



ETHERNETIP

Evaluation pratique

TP Noté

v(None)

IUT d'Annecy - Département GEII - 9 Rue de l'Arc en Ciel, 74940 Annecy

Ce travail pratique porte sur la mise en œuvre d'une cellule robotisée industrielle exploitant la solution **Uni-VALplc** du constructeur Stäubli pour l'automatisation d'un robot Scara. L'objectif est de réaliser le transfert automatique de gobelets entre des palettes circulant sur un convoyeur et de leur faire subir un traitement de surface.

Pour cela, vous serez amené, dans cette évaluation, à créer une partie du programme d'automatisme faisant se déplacer le robot puis à intégrer un pont roulant (nouvel élément) dans la cellule existante.

1 Présentation de la cellule robotisée

1.1 Rappels : Fonctionnement de la cellule

En fonctionnement normal, la cellule devra se comporter comme suit :

1. Attente en conditions initiales (A1) :

- Robot positionné au-dessus du guichet : (X=90, Y=-410, Z=150, RX=-180, RY=0, RZ=0) dans le repère « world »
- Flux de palettes passant (colonne verte)
- Bras sous puissance

2. Détection et saisie (F1) :

- Lorsqu'un gobelet est détecté, la palette en question est arrêtée sur le poste et le robot s'en saisit.
- Le robot dépose alors le gobelet sur la Cuve 0
- Après une temporisation de 2 **secondes**, le robot se saisit à nouveau du gobelet
- Le robot dépose alors le gobelet sur la prochaine palette disponible.

1.2 Organisation du projet

Les sections d'un cycle automate sont organisées de la manière suivante :

- **Sections de lecture** : ReadEthIpRobot (début de cycle API)
 - ◊ VAL_ReadAxesGroup : lecture état robot (CS9)
 - ◊ FB_ReadHmi : lecture commandes pupitre (M221)
- **Sections GEMMA** :
 - ◊ GM_GmmaPre : initialisation et configuration
 - ◊ GM_Gmma (SFC) : graphe des modes de marche
 - ◊ GM_GmmaPost : activation sections conditionnelles
- **Sections des modes** (conditionnelles) :
 - ◊ Graphes SFC : D2_Diagnostic, A5_RemiseEnRoute, A6_MiseEnCI, F1_ProductionNormale
 - ◊ Sections Post (ST) : actions combinatoires et scrutation des blocs fonctionnels
 - ◊ Utilisation de macro-étapes pour structurer les séquences complexes
- **Sections d'écriture** : WriteEthIpRobot (fin de cycle API)
 - ◊ FB_WriteHmi : commande voyants et écran (M221)
 - ◊ VAL_WriteAxesGroup : envoi commandes robot (CS9)

2 Réalisation d'un GRAFCET de prise/dépose

2.1 Un GRAFCET d'approche

Afin de généraliser la prise et la dépose d'objets par le robot, on propose d'écrire un GRAFCET de prise/dépose qui sera utilisé à chaque prise ou dépose d'un gobelet. Le squelette de ce GRAFCET vous a été donné au tableau et se trouve dans la section F1_EVAL_PriseDepose et sa section POST associée.

Pour réaliser ce comportement, on propose d'utiliser les variables de type T_CartesianPos suivantes

Type	Nom	Coordonnées	Description
T_CartesianPos	cartesianPos_Cuve0	(360,-160, 20, 180, 0, 0)	Position du gobelet sur la cuve 0
T_CartesianPos	cartesianPos_Cuve1	(450, 0, 20, 180, 0, 0)	Position du gobelet sur la cuve 1
T_CartesianPos	cartesianPos_Convoyeur	(90,-410, 50, 180, 0, 0)	Position du gobelet sur le convoyeur
T_Trsf	Trsf_Approche	(0, 0, 100, 0, 0, 0)	Translation d'approche
T_CartesianPos	cartesianPos_Destination		Variable d'échange entre GRAFCET principal et GRAFCET de mouvements.

Chaque bloc fonctionnel UniVALplc de mouvement est associé à une variable structurée reliée à ses entrées et sorties. L'appel des bloc est fait dans la section ScaraMoves qu'il est possible de consulter.

A chaque fois que l'on souhaite faire une prise ou dépose de gobelet, la stratégie proposée, pour l'étape correspondante, est la suivante :

- Dans le programme principale F1_ProductionNormale, une action **P1** est lancée
 - Elle définit la variable **cartesianPos_Destination** avec la position de destination voulue. *On utilisera ici les positions définies dans les variables définies ci-dessus.*
 - Elle positionne une variable **priseDepose** qui indiquera si le robot doit poser ou récupérer un gobelet.
- Le GRAFCET coopérant de la section F1_EVAL_PriseDepose se déclenche alors (*Condition de sortie de l'étape initiale à définir*)
 - Le point d'approche est calculé à partir du point de destination
 - Le robot est envoyé au point d'approche
 - Le robot descend jusqu'au point de destination
 - La prise ou dépose est effectuée
 - Le robot remonte
 - Le GRAFCET attend que l'étape du programme principal correspondante ne soit plus active
- Le programme principal se poursuit.

Manipulation 1 : Réalisation du GRAFCET de prise/dépose

Étape 1 Vérifier, et définir si besoin, les variables du tableau précédent.

Ouvrir la documentation afin de prendre connaissance du fonctionnement du bloc VAL_ShiftPoint.

Étape 2 Dans la section F1_EVAL_PriseDepose_Post, Appeler une instance de bloc VAL_ShiftPoint.

Étape 3 Dans la section F1_EVAL_PriseDepose_Post, utiliser ce bloc fonctionnel pour faire le calcul du point d'approche lors de l'étape F1_M_01

Étape 4 Provoquer le lancement du mouvement du robot pour aller au point d'approche.

Attention, il peut être judicieux de vérifier que le robot ne se trouve pas déjà au point d'approche avant de lancer ce mouvement.

Étape 5 Compléter le reste du programme pour suivre le comportement désiré

Étape 6 Faire **Valider** par l'enseignant

3 Intégration du poste de traitement de surface

3.1 Présentation du poste de traitement de surface

Le poste de traitement de surface est un système automatisé permettant de réaliser des opérations de traitement chimique ou galvanique sur des pièces. Il est composé d'un **pont roulant** motorisé qui déplace un **chariot** équipé d'un **crochet** pour immerger et transférer les pièces entre différentes **cuves de traitement**.

3.1.1 Architecture matérielle

- **Automate WAGO 750-8012** : contrôleur PFC200 gérant le poste
 - ◊ Adresse IP : 172.16.180.185
 - ◊ Communication : Ethernet/IP
 - ◊ Type : Little Endian
- **Chariot** (axe X - transversal) :
 - ◊ Positionnement au-dessus des cuves
- **Système de levage** (axe Z - vertical) :
 - ◊ Crochet pour suspendre les pièces
 - ◊ Montée/descente pour immersion dans les cuves
- **Cuves de traitement** :
 - ◊ Plusieurs cuves alignées contenant différents bains
 - ◊ Positions fixes et connues

3.2 Rappel de la configuration actuelle

3.2.1 Premier coupleur : NOC_ETHIP_CONVOYEUR

- **Adresse IP** : 172.16.180.180
- **Réseau** : 172.16.180.0/24 (classe B privée)
- **Capacité** : 128 mots en entrée, 128 mots en sortie
- **Équipement connecté** : PFC200 - Poste Guichet
 - ◊ Adresse IP : 172.16.180.190
 - ◊ Données produites (T→O) : 12 octets
 - ◊ Données consommées (O→T) : 4 octets
 - ◊ Variables créées :
 - guichet_IN : type T_guichet_IN
 - guichet_OUT : type T_guichet_OUT

3.2.2 Second coupleur : NOC_ETHIP_ROBOT

- **Adresse IP** : 192.168.0.1
- **Réseau** : 192.168.0.0/24 (classe C privée)
- **Capacité** : 128 mots en entrée, 128 mots en sortie
- **Équipements connectés** (ordre d'ajout) :
 1. **CS9 - Contrôleur robot Scara** (192.168.0.10)
 - ◊ Données produites (T→O) : 148 octets = 74 mots
 - ◊ Données consommées (O→T) : 124 octets = 62 mots
 - ◊ Variables créées :
 - cs9Scara_IN : type T_cs9Scara_IN
 - cs9Scara_OUT : type T_cs9Scara_OUT
 2. **M221 - Contrôleur E/S et HMI** (192.168.0.11)
 - ◊ Données produites (T→O) : 4 octets
 - ◊ Données consommées (O→T) : 40 octets
 - ◊ Variables créées :

- m221Scara_IN : type T_m221Scara_IN
- m221Scara_OUT : type T_m221Scara_OUT

3.2.3 Services Ethernet/IP

La communication avec le PFC200 du traitement de surface utilisera un service d'assemblage (IO-Scanning) similaire à celui mis en place pour le Guichet. Ce service échangera les données suivantes :

- **Données produites par le pont (T→O)** : état du pont (positions, capteurs, mode en cours)
- **Données consommées par le pont (O→T)** : commandes vers le pont (consignes de mouvement, démarrage cycle)

3.3 Questions

3.3.1 Configuration réseau

Question 1 Dessiner l'architecture réseau complète de la cellule robotisée incluant le poste de traitement de surface. Indiquer les adresses IP des différents équipements et automate.

3.3.2 Cartographie de la mémoire

Question 2 En analysant les adresses des appareils et en **justifiant votre réponse**, déterminer sur quel coupleur réseau (NOC) de l'automate M340 on configurera la liaison Ethernet/IP avec le contrôleur WAGO du traitement de surface ?

Le module de traitement de surface produit et consomme les données suivantes, dans cet ordre :

Données produites (T→O)	Type
Etat du chariot Arrêté, En mouvement, En Erreur, etc.	SINT
Cuve de destination en cours	USINT
Dernière position détectée	USINT
Code Erreur Chariot	SINT
Etat de la pince (0. Entre deux, 1. En haut, 2. En bas)	USINT
Code Erreur Pince	SINT
Données consommées (O→T)	Type
Commande Chariot Aller vers Cuve, Arrêter, etc.	SINT
Cuve de destination	USINT
Commande pince Monter, Arrêter, Descendre	USINT

Question 3 A partir des informations ci-dessus, établir la nouvelle cartographie mémoire. Elle devra contenir, au minimum :

- Les éléments de la configuration actuelle :
 - ◊ Les variables associées au premier coupleur (172.16.180.180) : adresses mémoires de base, taille mémoire en entrée/sortie, équipements connectés avec les adresses des variables associées
 - ◊ Les variables associées au second coupleur (192.168.0.1) : adresses mémoires de base, taille mémoire en entrée/sortie, équipements connectés avec les adresses des variables associées
- Les nouveaux éléments liés au poste de traitement de surface :
 - ◊ Les adresses de début et de fin pour les entrées (T→O)
 - ◊ Les adresses de début et de fin pour les sorties (O→T)

3.3.3 Déclaration des variables

Deux types ont été créés dans le fichier donné : `TFromPontTS` et `TToPontTS`. Ils sont créés pour récupérer les données en provenance et à destination du module.

Question 4 Donner les déclarations des deux variables suivantes avec les adresses à leur affecter.

- `fromPontTS` de type `TFromPontTS` pour représenter les données produites
- `toPontTS` de type `TToPontTS` pour représenter les données consommées

Donner la syntaxe complète de déclaration avec les adresses mémoire appropriées.

Note : Les types `TFromPontTS` et `TToPontTS` sont fournis

3.3.4 Programmation de la communication

Question 5 À quel moment du cycle automate la lecture des données produites par le module de traitement de surface doit-elle se faire ?

Question 6 À quel moment du cycle automate l'écriture des données consommées par le module doit-elle se faire ?

4 Intégration du poste de traitement de surface

4.1 Les blocs fonctionnels fournis

Le projet fournit les blocs fonctionnels suivants :

4.1.1 EVAL_ReadSurfaceTreatment

Ce bloc fonctionnel permet de lire les données provenant du poste de traitement de surface.

Entrées	Description
<code>Enable</code> : BOOL	Activation du bloc fonctionnel
<code>FromModule</code> : T_FromModule	Données provenant du module
Entrées/Sorties	Description
<code>mModule</code> : T_TreatmentModule	Structure de gestion du module

4.1.2 EVAL_WriteSurfaceTreatment

Ce bloc fonctionnel permet d'écrire les données vers le poste de traitement de surface.

Entrées	Description
<code>Enable</code> : BOOL	Activation du bloc fonctionnel
Sorties	Description
<code>ToModule</code> : T_ToPontTS	Données à envoyer au module
Entrées/Sorties	Description
<code>mModule</code> : T_TreatmentModule	Structure de gestion du module

4.1.3 EVAL_MoveCart

Ce bloc fonctionnel permet de commander les mouvements du pont roulant.

Entrées	Description
Execute : BOOL	Lancement du mouvement demandé
uiDestination : UINT	Cuve de destination du pont
Sorties	Description
xDone : BOOL	Mouvement terminé
xBusy : BOOL	Mouvement en cours
xError : BOOL	Erreur lors du mouvement
uiErrorCode : UINT	Code d'erreur
Entrées/Sorties	Description
Module : T_TreatmentModule	Structure de gestion du module

Un type de donnée `TctrlChariot` est également fourni au besoin.

Pour toute information complémentaire sur le fonctionnement de ces blocs, se référer à l'enseignant.

4.2 Travail demandé

Manipulation 2 : Ajout de la communication avec le poste de traitement de surface

Étape 7 Dans le *Navigateur de DTM*, ajouter le module de traitement de surface

Étape 8 Dans la section **Exclusive Owner** de cet élément, configurer la taille des entrées et de sorties.

Étape 9 Sur le NOC correspondant, dans la liste des équipements et dans la section correspondant à l'équipement ajouté, paramétrer l'adresse IP du module.

Étape 10 Faire **Valider** par l'enseignant

Manipulation 3 : Programmation de la communication avec le poste de traitement de surface

Étape 11 Déclarer les variables nécessaires à la communication avec le module de traitement de surface.

Étape 12 Ajouter les sections nécessaire à la lecture et l'écriture des données vers le module de traitement de surface. Les placer judicieusement dans le cycle automate.

Étape 13 Utiliser les blocs fonctionnels fournis pour réaliser la lecture et l'écriture des données.

Étape 14 Faire **Valider** par l'enseignant

Manipulation 4 : Programmation des mouvements du pont roulant

Étape 15 Dans la section **EVAL_AppelPontRoulant**, appeler le bloc fonctionnel **EVAL_MoveCart** pour réaliser les mouvements du pont roulant. Connecter une variable de type `TctrlChariot` à l'entrée/sortie **Module** du bloc fonctionnel.

Étape 16 Tester le fonctionnement du pont roulant à partir d'une table d'animation.

Étape 17 Faire **Valider** par l'enseignant

Manipulation 5 : Intégration des mouvements du pont roulant dans le programme principal

Lorsque le robot dépose le gobelet sur la cuve de traitement de surface, le pont roulant doit se déplacer vers la cuve 2, faire une pause de 2 secondes puis revenir à la cuve 0. Le robot attendra que le pont soit revenu à la cuve 0 avant de récupérer le gobelet et de le ramener au convoyeur.

Étape 18 Modifier le programme principal pour intégrer ce comportement.

Étape 19 Faire **Valider** par l'enseignant